Sales Analytics Dashboard

Frontend (React) Requirements Deck

This document specifies exactly what the frontend needs to do. It covers two primary surfaces — a conversational Chat Interface and a structured Generate Report flow — plus field-level behavior, dependencies, and states the frontend must handle. Backend (FastAPI + LangGraph + PostgreSQL) is already architected separately; this deck is scoped to UI/UX and frontend behavior only.

Prepared for	Frontend developer
Stack	React (frontend) → FastAPI → LangGraph orchestrator → PostgreSQL
Access model	Single user, no login/auth required
Target	Desktop only, not required to be mobile-responsive
Concurrent users	2-3 (per system architecture)

Contents

1. Application Structure
2. Chat Interface — Requirements
3. Generate Report — Field Specification
4. Field Dependency Logic
5. Generate Report — Screen States
6. Non-Functional Requirements
7. Open Items (to be filled in before build)
8. Suggested Upgrades (not required for v1)

1. Application Structure

The app has two primary views, switchable via a persistent top-level tab or sidebar (developer's choice of pattern, as long as switching is one click and state in each view is preserved when the user tabs away and back).

- **Chat** — a conversational interface for ad-hoc, pinpointed questions against the sales data.
- **Generate Report** — a structured form for building and exporting a formatted report (Excel or PDF).

These two views are connected: a chat answer can be escalated into a pre-filled report (see Section 2.3). They are not otherwise identical — chat is free-form Q&A, Generate Report is a fixed, exhaustive export.

2. Chat Interface — Requirements

2.1 Core behavior

- Standard chat UI: message list (user right-aligned, assistant left-aligned), a persistent input box pinned to the bottom, send-on-Enter.
- User can ask any natural-language question about the sales data (e.g. "which route did the most coffee volume last week").
- Assistant responses come from the backend (LangGraph orchestrator) as plain text, and may include a table and/or a chart depending on what the backend returns — the frontend should be able to render all three response types (text, table, inline chart image/embed) inside a single chat bubble.
- New chat / clear conversation action.
- Chat history persists for the session (does not need to persist across browser reloads unless noted otherwise in Section 7).

2.2 Loading & error handling

- Show a typing/processing indicator while waiting on the backend — some queries (large date ranges, novel-query sandbox) may take several seconds longer than a simple lookup.
- If the backend returns an error or the query can't be answered, show a plain-language inline message in the chat bubble itself (not a browser alert/toast) — e.g. "I couldn't find data for that — try narrowing the date range."
- If the backend is unreachable, show a distinct connection-error state, separate from a data/query error.

2.3 Chat-to-report handoff

When a chat answer maps to a structured query the report engine can also produce (i.e. the backend response includes report-compatible metadata — filters it used, such as date range, route, SKU), the assistant's chat bubble should show a **"Generate this as a report"** button below the answer.

- Clicking it switches the user to the Generate Report view with those same filters pre-filled (date range, days, route, SKU, customers — whatever the backend identified).
- The user can still edit any pre-filled field before generating, exactly as if they'd filled the form manually.
- This button only appears when the backend explicitly signals the response is report-eligible — the frontend does not need to guess this from the text itself; it will be a flag/field in the API response (to be confirmed by backend — see Section 7).

3. Generate Report — Field Specification

All eight fields below appear on one form. Fields marked **Required** block report generation until filled; all others have sensible defaults described in the table.

Field	Type / Control	Details & Behavior
1. Date Range *Required*	Two date pickers (From / To), format dd/mm/yyyy	Standard calendar picker, India date format. Both ends inclusive. No default — user must set a range before generating.
2. Days of Week	Multi-select chips or checkboxes: Mon, Tue, Wed, Thu, Fri, Sat, Sun	Filters the date range down to only the selected weekdays (e.g. "every Monday and Friday in March"). Default: all 7 days selected (i.e. no filtering) if left untouched.
3. Route	Multi-select dropdown or checkbox list, 7 fixed options	The 7 routes are a fixed, known list (not dynamic from DB) — hardcode as a constant unless told otherwise. Multi-select allowed. Default: all 7 routes selected if left untouched. This field's selection controls what appears in the Customers field (see Section 4).

Field	Type / Control	Details & Behavior
4. SKU	Two-step selector: (a) Branch chips — Coffee / Tea / Others / All, (b) dynamic SKU multi-select populated from DB based on branch(es) chosen	SKU branches (Coffee/Tea/Others) are fixed categories, but the individual SKUs within each branch come from the database and can change over time — this list must be fetched live from the backend, never hardcoded. Selecting 'All' branches loads SKUs from all three. Default: if no branch/SKU is touched, treat as 'All' (no SKU filtering).
5. Customers	Multi-select list (searchable), dynamically populated	Populated based on the Route(s) selected in Field 3 — only customers belonging to the currently selected route(s) are shown (customer-to-route mapping, including alias resolution, comes from the backend/DB). If the user changes the route selection after having picked customers, re-fetch the customer list and clear/re-validate the prior selection. Default: if left untouched, treat as ALL customers within the selected route(s) — do not require explicit selection.

Field	Type / Control	Details & Behavior
6. Generate Chart	Toggle switch, OFF by default	When switched ON, reveal a 'Chart Type' dropdown/select. **Leave this dropdown's option list empty/placeholder for now** — chart type options will be supplied separately before this goes to build. When OFF, no chart is included in the report output.
7. Output Format *Required*	Radio buttons: Excel / PDF	Exactly one must be selected before the report can be generated. No default pre-selected — force an explicit choice.
8. Description / Notes	Free-text multi-line textarea	Optional. Placeholder text like "Any specific instructions for this report (e.g. exclude returns, highlight top 5 customers)...". Passed through to the backend as-is for the AI to consider when building the report. No character limit needed on the frontend (backend will handle validation).

Generate Report button sits at the bottom of the form, disabled until all Required fields are valid (Date Range set, Output Format chosen). On click, show a progress/loading state on the button itself (spinner + "Generating...") until the backend returns the file — then trigger a file download and show a success confirmation.

4. Field Dependency Logic

Summarized relationships between fields, for quick reference while building:

- **Route → Customers:** changing Route re-fetches and re-renders the Customer list. Customers not belonging to the newly selected route(s) are removed from the current selection.
- **SKU Branch → SKU list:** changing the Coffee/Tea/Others/All branch selection re-fetches the specific SKU options shown below it.
- **Generate Chart toggle → Chart Type field:** Chart Type is only visible/interactive when the toggle is ON. When OFF, hide it entirely (don't just disable it).
- **Untouched fields → "all" semantics:** Days, Route, SKU, and Customers all follow the same rule — if the user never interacts with the control, treat it as if everything is selected, not as an empty filter. This should be a deliberate default state, not just an empty array being interpreted permissively by the backend.

5. Generate Report — Screen States

- **Idle:** form empty except defaults described above.
- **Validating:** Generate button disabled + inline hint near whichever Required field is missing, if the user tries to submit early.
- **Loading (post-submit):** button shows spinner, rest of form becomes read-only (prevents double submission / mid-flight edits).
- **Success:** file download triggers automatically; show a small toast/confirmation with the filename and format generated.
- **Empty result:** if the selected filters return zero matching rows, show a clear "No data found for these filters" message instead of downloading a blank file.
- **Error:** distinguish between a validation error (frontend-caught, e.g. To date before From date) and a backend/generation failure (show a retry option).

6. Non-Functional Requirements

- **Auth:** none required for v1 — single user, no login flow.
- **Responsiveness:** desktop-only; no mobile breakpoint work required.
- **Concurrency:** designed for 2-3 concurrent users — no need for heavy client-side caching/optimistic UI beyond normal good practice.
- **Data freshness:** the underlying database is synced daily from Excel. There is no requirement for the frontend to show real-time data, but if the backend exposes a "last synced" timestamp, display it somewhere unobtrusive (e.g. footer of the Generate Report view) so users know how current the data is.
- **Browser support:** latest Chrome/Edge is sufficient (internal tool).

7. Open Items (to be filled in before build)

The following are intentionally left open and will be confirmed in a follow-up before the developer starts (or can be treated as placeholders they build around):

- **Chart Type options** (Field 6) — list of chart/graph types to offer is still being finalized.
- **API contract** — exact endpoint names/payloads for chat, report generation, and the dynamic dropdowns (routes, SKUs, customers) will be provided separately by the backend team; frontend should build against mocked versions of these shapes in the meantime.
- **Chat-eligibility flag** — exact shape of the "this answer is report-eligible" signal from the backend (Section 2.3) to be confirmed.
- **Visual design / branding** — no brand guide provided yet; developer has discretion on visual styling for v1 unless a design file follows separately.

8. Suggested Upgrades (not required for v1)

Not part of the current build scope, but worth keeping in mind so the frontend architecture doesn't have to be reworked later if these get greenlit:

- **Saved report presets** — let users save a field combination as a named template for reuse.
- **Report history panel** — list of previously generated reports with re-download links.
- **Live chart preview** — render the selected chart in-browser before committing to export.
- **Searchable multi-selects** — for Route, SKU, and Customer fields, use type-to-filter rather than long static checkbox lists, since these lists are expected to grow.
- **Chat source citations** — show which filters/date range/table the chatbot used to produce an answer, so the user can trust it without re-verifying.

End of document.